

Assignment #03

Message Passing

Design and implementation report

Academic year 2025-2026 - Concurrent and Distributed Programming

This report documents the submitted implementation and its concurrency choices. The design follows the terminology and principles in merged.pdf and the assignment README.

1. Exercise 1 - Smart Home Alarm

The alarm is naturally modeled as actors because controller, keypad and sensors are autonomous components that communicate asynchronously. No actor exposes mutable state. AlarmController is the sole owner of the security state; Sensor and Keypad translate local input into immutable messages.

Current state	Event	Next state / action
Disarmed	Correct PIN	Exit Delay; start exit timer
Exit Delay	Timer	Armed
Armed	Sensor event	Entry Delay; start entry timer
Entry Delay	Correct PIN	Disarmed
Entry Delay	Timer	Alarm
Alarm	Correct PIN	Disarmed
Any state	Wrong PIN	No transition

Timer messages carry a generation number. Every state transition increments it, so a delayed timeout from an earlier state is ignored. Sensor events are effective only while Armed, exactly matching the specification. The design uses the actor macro-step property: one message handler completes before the next starts, eliminating races on the state machine.

The JUnit actor test enters the correct PIN, waits for the configured short exit delay and asks the actor for its state. The expected result is Armed. The sample application demonstrates the complete timeout path.

2. Exercise 2 - Odds-and-Evens

For $N = 2^m$ players, every round starts $N/2$ independent match goroutines. Every player is a persistent agent goroutine with its own pseudo-random state. A match sends requests to two players and receives their choices over unbuffered channels; it then sends the immutable winner value to the round coordinator.

There is no shared game state. The coordinator owns its local current-round slice, player agents own their random generators, and matches own their choices. All transfers cross channels. Matches in one round execute concurrently, while the next round begins only after all winner messages arrive. The number of rounds is m and the total number of matches is $N - 1$.

The Go test runs an eight-player tournament and checks that the winner identifier belongs to the original set. go test -race can additionally confirm that the message-passing implementation has no data races.